MOD-06: Fire Suppression Hardware Module

Detailed Technical Documentation

CSE 396 — Autonomous Fire Detection and Suppression Rover

Mehmet Alp ATAY (220104004020)

Adil Mert ERGÖRÜN (220104004048)

March 29, 2026 | Version 0.1

Contents

1 Module Overview **2**
- 1.1 Position in System Architecture . . . 2
- 1.2 Responsibilities . . . 2
- 1.3 File Location . . . 2
- 1.4 Dependencies . . . 2

2 Constants and Macros **3**

3 Data Types **3**
- 3.1 `suppression_status_t` — Return / Error Codes . . . 3
- 3.2 `suppression_state_t` — Module State Snapshot . . . 4
- 3.3 `suppression_cmd_t` — Command Structure . . . 4

4 Public API — Function Reference **5**
- 4.1 `suppression_init()` . . . 5
- 4.2 `suppression_activate()` . . . 5
- 4.3 `suppression_deactivate()` . . . 6
- 4.4 `suppression_is_active()` . . . 6
- 4.5 `suppression_get_state()` . . . 7
- 4.6 `suppression_apply_cmd()` . . . 7
- 4.7 `suppression_stop()` . . . 8

5 Timing and Hardware Constraints **8**
- 5.1 Cooldown Mechanism . . . 8
- 5.2 Default Burst Duration . . . 9
- 5.3 GPIO Signal Timing . . . 9

6 Inter-Module Protocol: MOD-07 → MOD-06 **9**
6.1 Emergency Stop Path 10

7 Error Handling Guide **10**
7.1 Defensive Coding Recommendations 11

8 Integration Example — Annotated C Code **11**
8.1 Annotated Walkthrough 13

9 Known Limitations and TODOs **14**

1 Module Overview

MOD-06 is the **Fire Suppression Hardware** module of the Autonomous Fire Detection and Suppression Rover. It is responsible for physically extinguishing a detected fire by operating a relay-driven DC water pump. The module abstracts all low-level hardware interactions (GPIO toggling, relay timing, cooldown enforcement) and exposes a clean, command-driven interface to the Decision and Autonomy module (MOD-07).

1.1 Position in System Architecture

MOD-07 (Autonomy SW)	$\xrightarrow{\text{suppression_cmd_t}}$	**MOD-06 (Suppression HW)**

MOD-06 receives `suppression_cmd_t` structs from MOD-07. It controls the relay GPIO pin (SUPPRESSION_RELAY_PIN) connected to the water pump circuit and maintains internal state (activation count, timestamps, cooldown).

1.2 Responsibilities

- Initialize relay GPIO pin and validate hardware readiness.
- Activate the water pump for a configurable duration.
- Enforce a minimum cooldown period between consecutive activations to protect relay hardware.
- Deactivate the pump immediately upon receiving a stop command.
- Expose current module state (active flag, timestamps, activation count).
- Execute structured commands issued by MOD-07.
- Safely power down hardware on system shutdown.

1.3 File Location

```
Header files/MOD 06/fire_suppression.h
```

1.4 Dependencies

- `<stdint.h>` — Fixed-width integer types (`uint8_t`, `uint16_t`, `uint32_t`).
- Arduino Core for ESP32 — `digitalWrite()`, `pinMode()`, `millis()` for GPIO and timing.
- No upstream module headers are `#included`; MOD-06 is a leaf hardware module.

2 Constants and Macros

Macro Name	Value	Purpose
`SUPPRESSION_PUMP_DURATION_MS`	`3000`	Default pump burst time in milliseconds (3 seconds). Used when the command's `duration_ms` field is zero.
`SUPPRESSION_COOLDOWN_MS`	`500`	Minimum interval between two consecutive activations (ms). Prevents relay chatter and premature wear.
`SUPPRESSION_RELAY_PIN`	`7`	GPIO pin number wired to the relay IN signal. Must match the physical board layout.

Table 1: MOD-06 compile-time constants defined in `fire_suppression.h`.

Note: `SUPPRESSION_RELAY_PIN` is the only hardware-specific constant. When porting to a different board, only this value needs to change.

3 Data Types

3.1 `suppression_status_t` — Return / Error Codes

This `typedef enum` is the return type for all public functions. It encodes success and specific failure modes.

Enumerator	Value	Meaning
`SUPPRESSION_OK`	`0`	Operation completed successfully.
`SUPPRESSION_ERR_INIT`	`-1`	Hardware initialisation failed (e.g., GPIO pin not configurable). Returned only by `suppression_init()`.
`SUPPRESSION_ERR_COOLDOWN`	`-2`	Activation request rejected because less than `SUPPRESSION_COOLDOWN_MS` has elapsed since last activation.
`SUPPRESSION_ERR_HARDWARE`	`-4`	Relay or pump hardware fault detected during operation. Returned by `suppression_deactivate()` if relay fails to open.

Table 2: `suppression_status_t` — all enumerators and their semantic meaning.

Warning: Error code `-3` is intentionally skipped (reserved for future use). Always compare against named constants, never raw integer values.

3.2 `suppression_state_t` — Module State Snapshot

A caller-owned struct populated by `suppression_get_state()`. Provides a consistent snapshot of the subsystem at any point in time.

Field	Type	Units / Range	Description
`is_active`	`uint8_t`	0 or 1	Non-zero while the pump relay is closed and water is flowing. Set by `suppression_activate()`, cleared by `suppression_deactivate()` or after the burst duration expires.
`last_activated_ms`	`uint32_t`	ms since boot	Timestamp recorded at the moment of the most recent successful activation. Used internally to enforce cooldown. Wraps after ≈49.7 days.
`total_activations`	`uint32_t`	count [0, 2^{32}-1]	Cumulative number of times the pump has been successfully activated since `suppression_init()` was called. Useful for wear monitoring and diagnostics.

Table 3: Fields of `suppression_state_t`.

3.3 `suppression_cmd_t` — Command Structure

Sent by MOD-07 (or MOD-08 for test injection). Contains all information needed to execute one suppression action.

Field	Type	Valid Range	Description
`activate`	`uint8_t`	0 or 1	Command intent: `1` = activate pump, `0` = deactivate pump immediately.
`duration_ms`	`uint16_t`	0 – 65535 ms	Duration override for pump-on time. A value of `0` instructs the module to use the compile-time default `SUPPRESSION_PUMP_DURATION_MS` (3000 ms).
`issued_at_ms`	`uint32_t`	ms since boot	Timestamp recorded by MOD-07 when the command was constructed. Enables MOD-06 to detect stale commands and measure command latency.

Table 4: Fields of `suppression_cmd_t`.

Note: When `activate == 0`, the `duration_ms` and `issued_at_ms` fields are still present but only `activate` is acted upon; the pump is deactivated regardless of duration.

4 Public API — Function Reference

4.1 suppression_init()

```
suppression_status_t suppression_init(void);
```

Purpose: Configures the relay GPIO pin as an output and drives it LOW (relay open, pump off). Resets internal state counters.

Attribute	Detail
Parameters	None
Returns	`SUPPRESSION_OK` (0) on success; `SUPPRESSION_ERR_INIT` (-1) if the GPIO cannot be configured.
Pre-conditions	Must be called once before any other MOD-06 function. Typically invoked in the system `setup()` routine.
Post-conditions	Relay is open (pump is off). `is_active == 0`, `total_activations == 0`, `last_activated_ms == 0`.
Side effects	Sets `SUPPRESSION_RELAY_PIN` to OUTPUT mode; asserts LOW.
Re-entrant	No — calling twice without `suppression_stop()` is not supported.

4.2 suppression_activate()

```
suppression_status_t suppression_activate(uint16_t duration_ms);
```

Purpose: Closes the relay (activates the pump) for the specified number of milliseconds. If `duration_ms` is 0, the default burst duration `SUPPRESSION_PUMP_DURATION_MS` (3000 ms) is used.

Attribute	Detail
Parameters	`duration_ms`: pump-on time in milliseconds; 0 ⇒ use default (3000 ms).
Returns	`SUPPRESSION_OK` on success; `SUPPRESSION_ERR_COOLDOWN` if cooldown has not elapsed; `SUPPRESSION_ERR_HARDWARE` if relay fails to close.
Pre-conditions	`suppression_init()` must have returned `SUPPRESSION_OK`.
Post-conditions	Relay closes; pump starts. `is_active == 1`. `last_activated_ms` is updated to current `millis()`. `total_activations` is incremented by 1.
Side effects	Drives `SUPPRESSION_RELAY_PIN` HIGH. Schedules automatic deactivation after `duration_ms`.
Cooldown check	Internally computes `millis() - last_activated_ms`. If less than `SUPPRESSION_COOLDOWN_MS` (500 ms), returns `SUPPRESSION_ERR_COOLDOWN` immediately without toggling the relay.
Re-entrant	No — calling while `is_active == 1` is undefined.

4.3 suppression_deactivate()

```
suppression_status_t suppression_deactivate(void);
```

Purpose: Immediately opens the relay and stops the pump, regardless of remaining burst duration.

Attribute	Detail
Parameters	None
Returns	`SUPPRESSION_OK` on success; `SUPPRESSION_ERR_HARDWARE` if the relay fails to open.
Pre-conditions	May be called at any time after `suppression_init()`. Safe to call even if pump is already off.
Post-conditions	Relay opens; pump stops. `is_active == 0`.
Side effects	Drives `SUPPRESSION_RELAY_PIN` LOW. Cancels any pending auto-deactivation timer.
Re-entrant	No.

4.4 suppression_is_active()

```
uint8_t suppression_is_active(void);
```

Purpose: Non-blocking query of pump status. Allows callers to poll the suppression state without copying the full state struct.

Attribute	Detail
Parameters	None
Returns	`1` if the pump relay is currently closed (pump running); `0` if the pump is idle.
Pre-conditions	`suppression_init()` must have been called.
Post-conditions	None — read-only query.
Side effects	None.
Re-entrant	Yes — safe to call from multiple contexts.

4.5 suppression_get_state()

```
suppression_status_t suppression_get_state(suppression_state_t *out);
```

Purpose: Copies the current internal module state into a caller-owned `suppression_state_t` struct for diagnostics, logging, or telemetry.

Attribute	Detail
Parameters	`out`: non-NULL pointer to a `suppression_state_t` buffer owned by the caller.
Returns	`SUPPRESSION_OK` always (assuming `out` is non-NULL and memory is accessible).
Pre-conditions	`out` must point to a valid, writable memory region of at least `sizeof(suppression_state_t)` bytes.
Post-conditions	All three fields of `*out` are populated with the current internal values.
Side effects	None — read-only operation.
Re-entrant	Yes.

Warning: Passing `NULL` as `out` is undefined behavior. The implementation does not perform a NULL check; callers are responsible for providing a valid pointer.

4.6 suppression_apply_cmd()

```
suppression_status_t suppression_apply_cmd(const suppression_cmd_t *
    cmd);
```

Purpose: High-level entry point intended for use by MOD-07 and MOD-08. Parses a `suppression_cmd_t` and dispatches to `suppression_activate()` or `suppression_deactivate()` accordingly.

Attribute	Detail
Parameters	`cmd`: non-NULL pointer to a `const suppression_cmd_t`.
Returns	Propagates the return value of whichever lower-level function is invoked. Returns `SUPPRESSION_OK` if command is applied successfully.
Pre-conditions	`cmd` must not be NULL. `suppression_init()` must have been called.
Post-conditions	Depends on `cmd->activate`: pump is either activated (for `cmd->duration_ms` ms) or immediately deactivated.
Side effects	Same as the underlying `suppression_activate()` or `suppression_deactivate()` call.
Stale command handling	If `millis() - cmd->issued_at_ms` exceeds a system-defined latency threshold, the command may be discarded and a warning logged (implementation-dependent).
Re-entrant	No.

4.7 `suppression_stop()`

```
void suppression_stop(void);
```

Purpose: Graceful shutdown. Opens the relay, ensures the pump is off, and releases any hardware resources held by the module. Called during system teardown.

Attribute	Detail
Parameters	None
Returns	`void`
Pre-conditions	May be called at any time. Safe even if the module was never initialised.
Post-conditions	Relay is open; pump is off. GPIO pin is set to INPUT (high-impedance) to minimise standby current.
Side effects	Drives `SUPPRESSION_RELAY_PIN` LOW then sets it to INPUT mode.
Re-entrant	No.

5 Timing and Hardware Constraints

5.1 Cooldown Mechanism

The cooldown guard is the most critical timing constraint in MOD-06. It prevents rapid successive relay toggling, which can cause contact wear, voltage spikes, and pump damage.

Parameter	Value / Notes
Cooldown period	500 ms (`SUPPRESSION_COOLDOWN_MS`)
Enforcement	Checked at entry of `suppression_activate()` using `millis() - last_activated_ms < SUPPRESSION_COOLDOWN_MS`.
Effect of violation	Returns `SUPPRESSION_ERR_COOLDOWN`; relay is **not** toggled.
Cooldown reset	`last_activated_ms` is updated **only** on successful activation.

5.2 Default Burst Duration

Parameter	Value / Notes
Default burst	3000 ms (`SUPPRESSION_PUMP_DURATION_MS`)
Override mechanism	Pass non-zero `duration_ms` to `suppression_activate()` or set `cmd.duration_ms` $\neq$ `0` in `suppression_apply_cmd()`.
Maximum useful burst	Hardware-limited by water reservoir capacity. No software cap is enforced.
Auto-deactivation	Implementation must schedule a timer or track elapsed time in `loop()` to deactivate after the burst duration.

5.3 GPIO Signal Timing

Parameter	Value
Relay GPIO pin	`SUPPRESSION_RELAY_PIN` = 7
Logic level (pump ON)	HIGH (3.3 V on ESP32 GPIO)
Logic level (pump OFF)	LOW (0 V)
Relay switch-on time	$\approx$10 ms (typical mechanical relay; hardware dependent)
Relay switch-off time	$\approx$5 ms (spring-return; hardware dependent)
ESP32 GPIO sink/source	Max 40 mA; relay IN draws $\leq$5 mA via opto-coupler

6 Inter-Module Protocol: MOD-07 $\rightarrow$ MOD-06

The following sequence describes the standard command-driven suppression cycle, as initiated by MOD-07 when the FSM enters the `EXTINGUISH` state.

Step 1. FSM enters EXTINGUISH state (MOD-07)
MOD-07's finite state machine determines that the rover has arrived at the target and begins suppression.

Step 2. MOD-07 constructs command struct

```
suppression_cmd_t cmd;
cmd.activate      = 1;
cmd.duration_ms   = 0;         /* use default: 3000 ms */
cmd.issued_at_ms  = millis();
```

Step 3. MOD-07 calls `suppression_apply_cmd(&cmd)`
MOD-06 receives the command, checks cooldown, closes the relay, updates `is_active` and `last_activated_ms`.

Step 4. Pump runs for 3000 ms
During this period, MOD-07 may poll `suppression_is_active()` to track progress. MOD-07 remains in `EXTINGUISH` state.

Step 5. Auto-deactivation
After the burst duration, MOD-06 opens the relay (`is_active` ← 0). Cooldown timer starts.

Step 6. MOD-07 transitions to VERIFY state
MOD-07 reads flame sensor data (via MOD-02) to confirm the fire is extinguished.

Step 7. Repeat or return to SEARCH
If flame is still detected after cooldown: repeat from Step 2.
If no flame detected: MOD-07 transitions to `SEARCH` state.

6.1 Emergency Stop Path

At any point, MOD-07 can issue an immediate stop:

```
suppression_cmd_t stop_cmd;
stop_cmd.activate     = 0;   /* deactivate */
stop_cmd.duration_ms  = 0;
stop_cmd.issued_at_ms = millis();
suppression_apply_cmd(&stop_cmd);
/* OR equivalently: */
suppression_deactivate();
```

Emergency: `suppression_deactivate()` bypasses the cooldown check and opens the relay immediately. It should be called on any system fault (battery low, sensor error, hardware fault).

7 Error Handling Guide

Error Code	Trigger Condition	Recommended Action
`SUPPRESSION_ERR_INIT`	GPIO pin configuration fails during `suppression_init()`.	Halt system startup; log the fault; do not proceed to main loop.
`SUPPRESSION_ERR_COOLDOWN`	`suppression_activate()` called within 500 ms of the previous activation.	Wait for cooldown to expire; retry after `SUPPRESSION_COOLDOWN_MS` ms. MOD-07 should track `last_activated_ms` to avoid repeated rejections.
`SUPPRESSION_ERR_HARDWARE`	Relay fails to open during `suppression_deactivate()`.	Critical fault. Log error; enter safe state (stop all motion); alert operator if telemetry is available.

Table 5: Error code guide for MOD-06 callers.

7.1 Defensive Coding Recommendations

- Always check the return value of `suppression_init()` before entering `loop()`.
- Store `suppression_state_t.last_activated_ms` in MOD-07 to precompute cooldown eligibility before issuing commands.
- If `SUPPRESSION_ERR_HARDWARE` is received, do not attempt to re-activate the pump without a full system reset.
- Never call `suppression_activate()` while `suppression_is_active() == 1`.

8 Integration Example — Annotated C Code

The following example demonstrates a complete lifecycle: initialization, command-driven activation from MOD-07, state polling, and graceful shutdown.

Listing 1: Complete MOD-06 integration example (Arduino / ESP32)

```c
#include "fire_suppression.h"
#include <Arduino.h>

/* --- System setup (called once) ---------------------------------------- */
void setup(void) {
    Serial.begin(115200);

    /* Step 1: Initialise MOD-06 */
    suppression_status_t st = suppression_init();
    if (st != SUPPRESSION_OK) {
        Serial.println("[MOD-06] FATAL: Init failed. Halting.");
        while (1); /* System halt */
    }
    Serial.println("[MOD-06] Initialised. Relay is open, pump is off.");
}

/* --- Main loop (called repeatedly) ------------------------------------- */
void loop(void) {
    /* Simulated trigger from MOD-07: flame detected, rover in position */
    bool flame_confirmed = true; /* Replace with actual MOD-07 decision */

    if (flame_confirmed) {

        /* Step 2: Build command struct (normally done inside MOD-07) */
```

```
        suppression_cmd_t cmd;
        cmd.activate     = 1;
        cmd.duration_ms  = 0;         /* 0 => use default 3000 ms */
        cmd.issued_at_ms = millis();

        /* Step 3: Apply command */
        suppression_status_t result = suppression_apply_cmd(&cmd);

        if (result == SUPPRESSION_OK) {
            Serial.println("[MOD-06] Pump activated.");

        } else if (result == SUPPRESSION_ERR_COOLDOWN) {
            Serial.println("[MOD-06] Cooldown active, skipping.");

        } else if (result == SUPPRESSION_ERR_HARDWARE) {
            Serial.println("[MOD-06] HARDWARE FAULT. Entering safe
               state.");
            suppression_stop();
            while (1); /* Halt for safety */
        }

        /* Step 4: Poll until deactivated (non-blocking with millis()
           ) */
        uint32_t wait_start = millis();
        while (suppression_is_active()) {
            if (millis() - wait_start > 5000UL) {
                /* Safety timeout: force deactivation */
                suppression_deactivate();
                Serial.println("[MOD-06] Safety timeout; pump force-
                   stopped.");
                break;
            }
            delay(50);
        }

        /* Step 5: Read state for diagnostics */
        suppression_state_t state;
        suppression_get_state(&state);
        Serial.print("[MOD-06] Total activations: ");
        Serial.println(state.total_activations);
    }

    /* --- Other system tasks run here --- */
    delay(100);
}

/* --- System teardown (e.g., called on power-off signal)
```

```
   --------------- */
69 void teardown(void) {
70     suppression_stop();
71     Serial.println("[MOD-06] Shutdown complete.");
72 }
```

8.1 Annotated Walkthrough

Line 14 `suppression_init()` must be the first MOD-06 call. A non-zero return value is a fatal hardware error.

Line 31–34 The `suppression_cmd_t` struct is the standard interface between MOD-07 and MOD-06. Setting `duration_ms = 0` defers to the compile-time default.

Line 37 `suppression_apply_cmd()` dispatches to either `suppression_activate()` or `suppression_deac` based on `cmd.activate`.

Line 50–56 A safety watchdog using `millis()` prevents the pump from running indefinitely if the auto-deactivation timer fails.

Line 59–63 `suppression_get_state()` is called after every cycle for telemetry. `total_activations` can be used to schedule preventive maintenance.

Line 71 `suppression_stop()` sets GPIO to input mode, reducing standby current draw.

9 Known Limitations and TODOs

#	Limitation	Recommended Resolution
1	No water level sensor. Module cannot detect an empty reservoir.	Add a float sensor on MOD-06 hardware and expose a `suppression_is_reservoir_empty()` function.
2	Auto-deactivation relies on polling in `loop()`. A timing overrun in another task can extend the pump burst.	Migrate to a FreeRTOS timer callback for hardware-accurate deactivation.
3	`SUPPRESSION_RELAY_PIN` is a compile-time constant, requiring recompilation for different boards.	Refactor to a runtime-configurable parameter passed into `suppression_init(uint8_t relay_pin)`.
4	No interrupt-driven GPIO feedback confirming relay state.	Add a feedback pin from the relay NO/NC contacts to verify actual relay state on `deactivate()`.
5	`millis()` wraps at ≈49.7 days; cooldown computation will fail across a wraparound event.	Use modular arithmetic: `(uint32_t)(millis() - last_activated_ms)` to handle wraparound correctly.
6	No logging or telemetry interface defined in v0.1.	Define a callback `suppression_set_log_cb()` to allow the system to attach a serial or MQTT logger.

Table 6: Known limitations and recommended resolutions for MOD-06 v0.1.